

ReCache: Efficient KV Cache Reuse and Compression for Tool-Augmented LLM Agents

Yichu Fang^{1,2}, Sitong Wei³, Haozhe Hu^{1,2}, Xiaoyu Shen^{2*}

¹Shanghai Jiao Tong University

²Eastern Institute of Technology, Ningbo

³Xi'an Jiaotong University

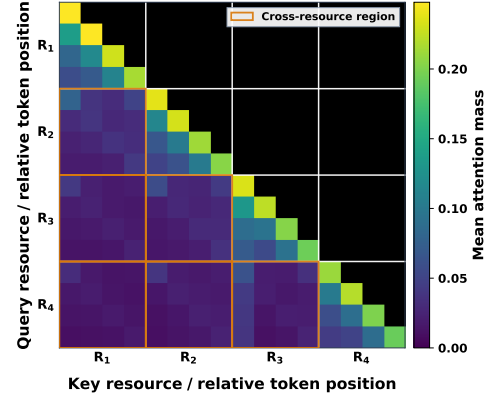
fy_chu@sjtu.edu.cn xyshen@eitech.edu.cn

Abstract

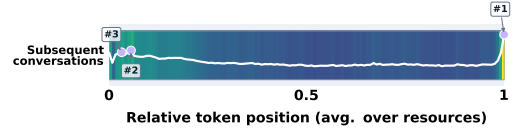
Agentic language models repeatedly encode tool and skill schemas that recur across requests in different combinations and orders, preventing standard prefix caching from reusing their key-value (KV) states. We introduce **ReCache**, a framework for independently caching resource representations while reducing their inference-time computational and memory overhead. Resource-wise attention removes cross-resource interactions and assigns resource-local positions, producing composition-invariant KV blocks. ReCache then restricts resource visibility to contribution-selected layer-KV-head-group routes and retains only invocation-critical fields through structural and semantic pruning. We evaluate ReCache on a benchmark assembled from seven public tool- and skill-use datasets, including resource-disjoint tests. Resource-wise attention matches dense invocation performance (82.3% versus 82.4% Inv-F1) while providing a $3.655\times$ time-to-first-token speedup. The complete framework reduces allocated KV-tensor memory by 92.43% and accelerates attention by $1.423\times$. These results show that separating reusable schema encoding from selective resource access substantially reduces agentic inference costs with limited effectiveness loss. The code is available at <https://github.com/EIT-NLP/ReCache>.

1 Introduction

Recent advances have extended large language models (LLMs) into the core reasoning module of agentic systems, where they address user requests by dynamically invoking external tools and skills (Schick et al., 2023; Liu et al., 2024a; Li et al., 2026; Xu and Yan, 2026). As the set expands, agentic systems commonly adopt *progressive disclosure* and retrieve only task-relevant schemas during execution (Shi et al., 2025; Zheng et al., 2026). We



(a) Causal attention among resources (relative-position bins). Orange boxes mark cross-resource blocks.



(b) Attention from subsequent conversation tokens onto resources, normalized and aligned by relative position.

Figure 1: Cross-resource attention on Qwen3-4B.

refer to each tool or skill schema as a *resource*. Although retrieval limits the active context, the same resource may recur across requests in different combinations and orders, causing repeated prefill computation for unchanged information.

KV-cache reuse provides a natural way to amortize this repeated computation. However, standard prefix caching requires reused content to form an identical prefix, which rarely holds for dynamically composed resource contexts (Ye et al., 2025; Zhang et al., 2026). Recent approaches extend KV reuse to modular or position-independent contexts (Gim et al., 2024; Hu et al., 2024; Yao et al., 2025; Yang et al., 2025c), but often require positional correction or selective recomputation to approximate full-

*Corresponding author

context KV states. Meanwhile, transformers can jointly encode semantic and positional dependencies (Vaswani et al., 2017; Shaw et al., 2018; Su et al., 2024), and the importance of positional information varies across tasks, with some studies showing that certain tasks can maintain performance under reduced or altered positional signals (Wang et al., 2021; Haviv et al., 2022; Kazemnejad et al., 2023). In resource invocation, preserving resource-internal semantics may therefore matter more than preserving global resource order. Supporting this intuition, Figure 1a shows that Qwen3-4B exhibits substantially stronger attention within individual resources than across resources.

Based on this observation, we introduce **ReCache**, a resource-level KV reuse and compression framework for dynamically retrieved tools and skills. Its core mechanism, *resource-wise attention*, removes attention between distinct resource blocks and resets positional indices within each block. The resulting KV representation is independent of neighboring resources and their absolute positions, enabling resource caches to be constructed and reused independently. Although this representation differs from conventional full-context prefill, lightweight fine-tuning adapts the model to the resulting attention and positional patterns while preserving reliable resource invocation.

While resource-wise attention enables independent KV reuse, storing and transmitting complete resource caches can still be costly as resource length and scale grow. ReCache therefore compresses resource caches from both *structural* and *semantic* perspectives. Structurally, transformer layers and KV head groups contribute unevenly to model predictions (Michel et al., 2019; Voita et al., 2019; Dehghanighobadi and Fischer, 2026). ReCache ranks layers and head groups by their marginal contribution to prediction loss and retains resource visibility only along the most important routes. This contribution-based criterion exploits the distinct sparsity patterns across sequential layers and parallel KV head groups, providing a more reliable measure of resource utility than attention-mass selection. Semantically, Figure 1b shows that the final suffix token receives the highest attention mass from subsequent conversation tokens, suggesting that it naturally aggregates information from preceding fields (Muennighoff, 2022). This observation motivates retaining the suffix token as a compact semantic anchor, similar in spirit to summary-based compression methods that aggre-

gate prompt or segment information into summary tokens (Mu et al., 2023; Chevalier et al., 2023; Zhang et al., 2024). However, unlike general text, tool and skill schemas must preserve exact interface information, including resource identifiers, argument names, and parameter constraints; invalid names and arguments are common sources of invocation failure (Song et al., 2023; Qin et al., 2024; Hao et al., 2023). ReCache therefore retains these critical fields alongside the suffix token for semantic aggregation.

To evaluate resource reuse and generalization systematically, we construct a unified tool- and skill-use benchmark from seven public datasets. We first apply a diversity-first sampling strategy to maximize resource diversity across samples, followed by filtering of invalid traces containing hallucinated resource usages. The resulting benchmark contains both in-distribution and resource-disjoint out-of-distribution splits, enabling evaluation on both known and unseen resources.

Experiments with Qwen3 backbones show that resource-wise attention differs from Dense, the standard dense-attention baseline, by at most 0.2% across effectiveness metrics while providing a $3.655\times$ time-to-first-token (TTFT) speedup. With structural and semantic pruning, ReCache reduces allocated KV-tensor memory by 92.43% and accelerates attention by $1.423\times$. Under matched structural budgets, contribution-based selection preserves higher invocation effectiveness than attention-based and layer-asymmetric alternatives, particularly on unseen resources. Direct comparisons further show that field-aware semantic pruning retains better performance than generic text-compression baselines under the evaluated configurations. Across increasing resource lengths, ReCache maintains nearly constant TTFT, TPOT, and attention latency while capping allocated KV-tensor memory at 0.03 GiB.

Our contributions are threefold: (1) **ReCache** provides a resource-level KV reuse and compression framework for dynamically retrieved tools and skills; (2) resource-wise attention, contribution-guided structural pruning, and field-aware semantic pruning jointly reduce redundancy in cache construction, layer-head-group access, and token retention; and (3) a unified tool- and skill-use benchmark with resource-disjoint out-of-distribution evaluation supports systematic analysis of efficiency and invocation performance.

2 Related Work

Agentic Resources. Tool-augmented language models invoke external functions described by structured schemas. Recent benchmarks also evaluate function calling over large API pools and multi-turn interactions (Li et al., 2023; Liu et al., 2024a; Qin et al., 2024; Liu et al., 2024b; Wang et al., 2024a). Agent systems also use natural-language skills that package reusable procedures with invocation metadata (Xu and Yan, 2026; Jiang et al., 2026; Li et al., 2026). As these resource pools grow, the corresponding retrieval selects a task-relevant subset before inference (Shi et al., 2025; Zheng et al., 2026). Retrieval reduces the number of resources in the prompt, but it does not eliminate repeated encoding of recurring resources.

Reusable Caching. Prefix caching reuses KV states for requests sharing an identical prefix (Kwon et al., 2023; Zheng et al., 2024; Yu et al., 2025). However, its applicability to dynamically composed resources remains limited, as reordered or recombined segments may differ from the positional configurations under which their KV states were constructed. Position-independent methods address this mismatch through various strategies. EPIC (Hu et al., 2024) and CacheBlend (Yao et al., 2025) preserve original positional encodings while selectively recomputing salient tokens to improve cache integration. In particular, CacheBlend further corrects positional information across cached KV states to mitigate position mismatch during reuse. KVLink (Yang et al., 2025c) instead decouples positional information from cached content and compensates for positional shifts during reuse. KVCOMM (Ye et al., 2025), on the other hand, uses the entropy of embedding-based anchor weights to assess cache shareability. Nevertheless, the necessity of explicit positional information can vary across tasks. NoPo (Haviv et al., 2022) and NoPE (Kazemnejad et al., 2023) demonstrate that Transformers can recover positional knowledge without explicit positional encodings in various downstream tasks, while Wang et al. (Wang et al., 2021) further show that positional information exhibits task-dependent importance across classification and span prediction.

KV-Cache Compression. KV-cache compression exploits structural redundancy across tokens, layers, and attention heads. H2O evicts low-importance states under a uniform layer budget,

whereas DepthKV assigns layer-dependent retention budgets (Zhang et al., 2023; Dehghanighobadi and Fischer, 2026). DuoAttention classifies heads by their attention roles, and SPEED restricts prompt-KV visibility by depth while preserving full-depth decoding states (Xiao et al., 2025; Oh et al., 2026). These studies establish the importance of multidimensional sparsity, but their selection criteria are not optimized for resource invocation. In addition, semantic compression reduces the number of retained context tokens. Learned approaches encode spans into compact summary states (Mu et al., 2023; Chevalier et al., 2023; Zhang et al., 2024), while LLMLingua and LongLLMLingua apply query-aware token filtering (Jiang et al., 2023, 2024). ChunkKV further retains contiguous segments to preserve local coherence (Liu et al., 2026). These techniques effectively compress natural-language contexts while retaining salient information, but resources contain structured identifiers and field relationships that directly determine invocation behavior, motivating compression strategies tailored to field-level semantics.

3 ReCache

Problem Setup Given a dataset $\mathcal{D}$, each instance $(X, Y) \in \mathcal{D}$ consists of an agentic input sequence X and the target resource invocations Y required to satisfy a user request, including tool and skill invocations. The input sequence comprises the system instruction, user query, retrieved resources $\mathcal{R} = \{R_1, R_2, \dots, R_N\}$, and preceding conversational history when available in multi-turn interactions. Each resource $R_i = (t_{i,1}, \dots, t_{i,D_i})$ is represented as a token sequence of length D_i describing its functionality, including resource identifiers, argument definitions, and associated metadata.

The objective is to predict the required invocations Y conditioned on X . As illustrated in Figure 2, ReCache improves resource-processing efficiency through three progressive stages. Resource-wise attention removes unnecessary resource-resource interactions to enable independent KV representations; structural pruning restricts resource visibility to selected layer-KV-head-group routes; and semantic pruning reduces the visible resource length by retaining only critical fields.

Resource-Wise Attention Retrieval-Augmented Generation (RAG) demonstrates that disjoint retrieved passages can be encoded independently without mutual cross-attention (Izacard and Grave,

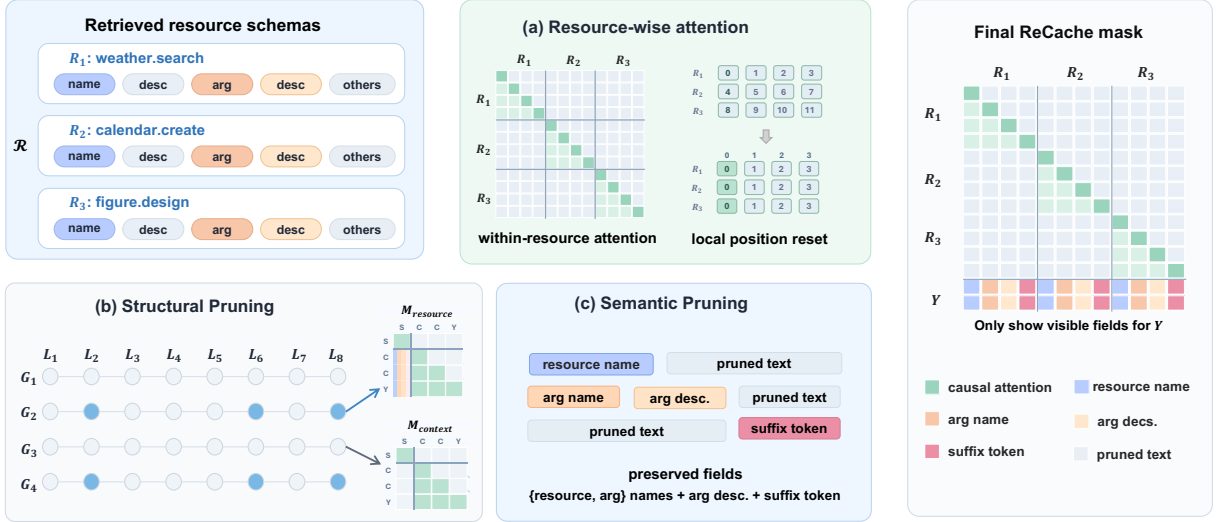


Figure 2: ReCache constructs reusable KV blocks for each R_i and progressively reduces their subsequent cache retention and access. (a) Resource-wise attention removes inter-resource attention and resets positions from zero within each R_i . (b) Structural pruning exposes resource KV states (using M_{resource}) only through selected layer (L_i)-KV-head-group (G_i) routes Ω^* (highlighted in blue). The remaining routes use M_{context} . (c) Semantic pruning retains resource names, argument (arg) names and descriptions (desc.), and the final suffix token. The final mask combines these decisions, ensuring that Y attends exclusively to retained semantic fields within assigned routes during decoding.

2021; Lin et al., 2025). Extending this principle, agentic resources are also retrieved (Shi et al., 2025) and primarily rely on internal semantics, allowing their KV caches to be constructed and reused independently of retrieval combinations or positional orderings.

Defining $D_{\mathcal{R}} = \sum_i D_i$ as the total resource length, resource-wise attention removes cross-resource dependencies during cache construction. As illustrated in Figure 2(a), tokens within each resource attend only to the shared prefix and their local resource tokens, while attention links between different resources are eliminated. The resource-resource attention is thus reduced from $O(D_{\mathcal{R}}^2)$ to $O(\sum_i D_i^2)$, allowing each R_i to be independently represented and cached.

To achieve full reusability, resource KV representations must also remain positionally invariant. Following prior studies on task-dependent positional importance (Wang et al., 2021; Haviv et al., 2022; Kazemnejad et al., 2023), we hypothesize that absolute placement among retrieved resources provides limited benefit for resource invocation. Therefore, we adopt a resource-local positional layout, where each resource follows an identical relative position scheme that preserves internal token continuity while removing dependence on global resource ordering. Specifically, during cache construction, ReCache assigns resource-local positional indices $\text{pos}(t_{i,j}) = j$ while preserving the

original positional encodings of the surrounding context. Consequently, each resource receives identical positional embeddings regardless of its placement among retrieved items. While frameworks like EPIC (Hu et al., 2024) incorporate position resetting as part of a broader compilation pipeline, empirical ablations confirm that this re-indexing is sufficient to preserve task effectiveness.

Structural Pruning Transformer layers and attention heads exhibit substantial redundancy (Michel et al., 2019; Voita et al., 2019; Men et al., 2024). ReCache exploits this structural sparsity by restricting resource-context visibility to influential layer-KV-head-group routes. Following objective-driven selection methods (Xiao et al., 2025; Dehghanigobadi and Fischer, 2026), route importance is measured by its marginal reduction in invocation loss.

Formally, let $[L] = \{1, \dots, L\}$ and $[G] = \{1, \dots, G\}$ denote the sets of transformer layers and KV head groups under grouped-query attention (GQA), respectively. A structural routing configuration is defined by a subset $\Omega \subseteq [L] \times [G]$, where each $(l, g) \in \Omega$ represents a layer-head-group route with access to resource KV states constructed by resource-wise attention. As illustrated in Figure 2(b), routes in Ω use M_{resource} to attend to the retained resource tokens and the conversational context. Routes outside Ω use M_{context} , which structurally prunes all resource tokens from their

attention scope while preserving standard causal attention over non-resource tokens.

The key challenge is to identify a compact Ω without degrading task performance. Contrary to DepthKV (Dehghanigobadi and Fischer, 2026), which evaluates layer importance by pruning one layer at a time from a dense configuration, ReCache applies leave-one-in analysis to both layers and KV head groups¹. Starting from $\Omega = \emptyset$, where conversation tokens cannot access resource states, it measures the reduction in invocation loss obtained by activating one layer or head group.

We denote the held-out selection set by $\mathcal{D}_h$ and the span of the target response Y by $\mathcal{T}(Y)$. For each $(X, Y) \in \mathcal{D}_h$, the invocation loss under configuration Ω is

$$\ell_\Omega(X, Y) = -\frac{1}{|\mathcal{T}(Y)|} \sum_{t \in \mathcal{T}(Y)} \log p_\theta^\Omega(y_t | y_{<t}, X),$$

where p_θ^Ω denotes the model distribution induced by Ω . Restricting the objective to invocation-related positions aligns route selection with resource identification and argument generation, while preventing unrelated response tokens from dominating the score. The selection objective $\mathcal{J}(\Omega)$ is then the average of $\ell_\Omega(X, Y)$ over $\mathcal{D}_h$.

For layer l , we activate all associated KV head groups using $\Omega_l = \{l\} \times [G]$ and define the layer contribution as

$$s_l = \mathcal{J}(\emptyset) - \mathcal{J}(\Omega_l).$$

A positive s_l indicates that resource visibility at layer l improves prediction performance relative to $\Omega = \emptyset$. Head-group contributions s_g are computed analogously using $\Omega_g = [L] \times \{g\}$. For each structural dimension, positive contributions are normalized into contribution weights:

$$w_i = \frac{\max(s_i, 0)}{\sum_j \max(s_j, 0)}$$

where i denotes a layer or KV head group. A larger w_i indicates a greater isolated contribution to resource-dependent prediction, and guides the selection of candidate structural budgets.

¹In preliminary experiments, leave-one-out scoring produced only small loss differences between structural units, whereas attention-mass ranking yielded lower invocation effectiveness than contribution-based selection. The latter result may stem from heterogeneous sensitivity across structural dimensions and token positions (Ding et al., 2024).

Specifically, given layer and head-group budgets K_L and K_G , ReCache retains the highest-weight units according to w_i :

$$\begin{aligned} \mathcal{L}^* &= \text{TopK}_{K_L}(\{w_l\}), \\ \mathcal{G}^* &= \text{TopK}_{K_G}(\{w_g\}), \end{aligned}$$

and constructs the final routing configuration as

$$\Omega^* = \mathcal{L}^* \times \mathcal{G}^*.$$

During inference, only routes in Ω^* expose resource KV states to conversational texts to reduce unnecessary computation.

Semantic Pruning While resource-wise attention enables resource-level cache reuse and structural pruning reduces unnecessary resource visibility, each activated route still processes the full D_i tokens within independent KV blocks. These tokens are inherently structural (Qin et al., 2024; Bai et al., 2026) and span heterogeneous fields, including executable identifiers, parameter constraints, descriptions, and formatting elements, with varying contributions to invocation performance, motivating field-aware compression. Since resource invocation failures are predominantly associated with invalid names and incorrect arguments (Song et al., 2023; Qin et al., 2024; Hao et al., 2023), ReCache performs semantic pruning along the token dimension to preserve invocation-critical components while removing redundant content.

For each R_i , ReCache retains tokens from three fields: the resource name, argument names, and argument descriptions. These fields support resource identification, parameter-interface alignment, and argument-value constraints, respectively. Ablation results show that preserving these three fields incurs only minor performance degradation².

Additionally, the final token is preserved as a suffix representation, whose hidden state aggregates preceding resource information. This design is related to summary-based compression approaches (Mu et al., 2023; Zhang et al., 2024), but instead of introducing new summary tokens, it reuses the final token of R_i as a naturally occurring boundary representation whose causal state

²Preliminary experiments with attention-based top- k token selection (while preserving resource and argument names) provide limited improvements, suggesting that complete argument descriptions contain essential semantic information. Chunk-based selection strategies following Zhang et al. (2024); Chevalier et al. (2023) require substantially more retained segments while providing limited interpretability.

can condition on all preceding fields (Muennighoff, 2022; Wang et al., 2024b; Zhuang et al., 2024)³. The resulting subsequence $\tilde{R}_i$ has an effective length of $\tilde{D}_i \leq D_i$.

The resulting compact KV representations remain independently reusable, while conversational texts on routes in Ω^* attend only to the retained tokens. As illustrated by the *Final ReCache Mask* in Figure 2, this selective visibility further reduces resource–context attention overhead.

4 Experimental Setup

4.1 Datasets

We evaluate ReCache on a unified resource-use dataset covering both tools and skills. Since existing datasets typically focus on particular resource types or interaction patterns, we combine seven sources to broaden resource and scenario coverage, including ToolACE (Liu et al., 2024a), API-GEN (Liu et al., 2024b), ToolMind (Yang et al., 2025b), ToolRet (Shi et al., 2025), Toucan (Xu et al., 2025), SkillRouter (Zheng et al., 2026), and WildToolBench (Yu et al., 2026).

Direct aggregation, however, introduces several confounding factors. Synthetic traces may contain malformed invocations or use resources absent from the provided set. More subtly, resource knowledge can be encoded into model parameters through learned embeddings or special tokens (Hao et al., 2023; Wang et al., 2025). ToolOmni further observes that parameter memorization generalizes poorly to unseen and evolving resources (Huang et al., 2026). Frequently repeated resource names or exact candidate subsets may consequently allow the model to recall familiar interfaces rather than interpret the schema supplied at inference time. We therefore retain only parseable and grounded resource calls and adopt **diversity-first sampling**, which prioritizes newly observed called resources and candidate-set configurations before satisfying source-level quotas.⁴

The final training dataset $\mathcal{D}$ contains 49,424 examples, while a subset $\mathcal{D}_{\text{subset}}$ with 5,000 examples is used for preliminary experiments. Evaluation is conducted on two test sets with 1,000 examples

³Preserving multiple trailing tokens (e.g., the last 10 or 20 tokens) provides limited additional gains.

⁴An initial direct-aggregation strategy exhibited strong frequency bias toward recurring resources and duplicated candidate subsets. Random downsampling failed to alleviate this imbalance, motivating the two-stage diversity-first selection detailed in Appendix A.

each: an in-distribution split $\mathcal{T}_{\text{IND}}$, where the involved resources are observed during training, and an out-of-distribution split $\mathcal{T}_{\text{OOD}}$, where the resources are unseen in $\mathcal{D}$.

4.2 Evaluation Metrics

Effectiveness We evaluate effectiveness using the turn-averaged resource-invocation F1 (**Inv-F1**), where each turn may involve multiple parallel resource invocations for one instance. This metric measures whether the model can generate the complete set of required invocations for a given query. A predicted invocation is considered correct only if both the resource name and all associated arguments exactly match the corresponding gold invocation. Beyond end-to-end performance, we further analyze model behavior from two complementary perspectives. Resource identification (Resource ID) is evaluated using precision (**ID-P**), recall (**ID-R**), and turn-averaged F1 (**ID-F1**) over resource names, reflecting the ability to identify relevant resources. We additionally report the resource hallucination rate (**Halluc.**), defined as the proportion of predicted resource names that do not exist in the available resource set, which captures failures caused by generating invalid or unavailable resources. All metrics are reported in percentages (%).

Efficiency. We use time-to-first-token (**TTFT**) to quantify online prefill latency under a cache hit, including request-local KV materialization but excluding offline cache construction. We further report time-per-output-token (**TPOT**) to measure end-to-end decoding latency, and attention latency (**Attn.**) to expose the decoding gains attributable specifically to reduced attention computation, which can be obscured in TPOT by unchanged projection and feed-forward costs. Finally, **Mem.** measures the resource reduction in memory allocated to KV-cache tensors.

4.3 Backbone Models

We perform full fine-tuning using Qwen3 variants (Yang et al., 2025a), with Qwen3-4B (Q_l) as the primary backbone. Qwen3-1.7B (Q_s) serves as an auxiliary backbone for analyzing the sensitivity of structural budgets to model scale. We train the primary Qwen3-4B model on an NVIDIA A800 GPU and run the preliminary Qwen3-1.7B experiments on an NVIDIA RTX 5000 GPU.

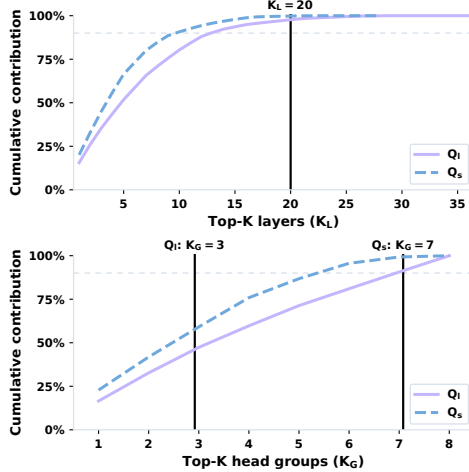


Figure 3: Structural contribution coverage in Q_l and Q_s . Cumulative leave-one-in contributions of layers (top) and KV head groups (bottom); vertical lines mark the selected budgets.

5 Structural Budget Analysis

Rather than treating structural budgets as unexplained hyperparameters, we derive them from how resource information is used across the network. Motivated by the non-uniform structural sensitivity observed in layer and KV-cache pruning (Men et al., 2024; Dehghanigobadi and Fischer, 2026), we use the leave-one-in scores from Section 3 to profile layers and KV head groups, then validate the resulting budgets through end-to-end ablations, compared with the full-visibility configuration Ω_{full} after applying resource-wise attention. For $\Omega = \mathcal{L} \times \mathcal{G}$, structural sparsity is

$$\rho(\Omega) = 1 - \frac{|\mathcal{L}||\mathcal{G}|}{LG},$$

which measures the fraction of routes without resource visibility. We also introduce contribution coverage (CC) to quantify the preserved structural capacity under a routing budget. CC is computed as the cumulative contribution weight w_i of the retained top- k layers or KV head groups.

Layer contributions. Figure 3 shows that layer contributions are highly concentrated: both cumulative curves rise rapidly and saturate near $K_L = 20$, where the retained layers capture 97.7% and 99.8% CC for Q_l and Q_s , respectively. This non-uniformity agrees with prior layer-pruning analyses (Fan et al., 2020; Men et al., 2024) and indicates that exposing resource states in every layer is unnecessary. We therefore use $K_L = 20$ as the shared

layer budget and jointly validate it with the selected head groups against Ω_{full} .

Head-group contributions. Head-group profiles differ by model scale. In Q_l , contributions are relatively flat, yet three groups retain near-full invocation performance despite covering only 47.3% CC. This discrepancy suggests that resource-related functions are not concentrated in a single head group but distributed across multiple groups, allowing certain groups to be removed with limited impact (Michel et al., 2019; Voita et al., 2019). In contrast, Q_s requires seven groups, which cover 99.3% CC and recover Inv-F1 to within 0.7 points of Ω_{full} . The smaller model therefore distributes this computation across more parallel groups, whereas the larger model offers greater substitutability, echoing the functional specialization of KV heads observed in long-context retrieval (Wu et al., 2024; Xiao et al., 2025). Table 1 summarizes the resulting budgets, structural sparsity, and invocation performance relative to Ω_{full} .

Model	K_L/L	K_G/G	CC _L	CC _G	ρ	$\Delta\text{Inv-F1}$ (%)
Q_l	20/36	3/8	97.7	47.3	79.2	-0.2
Q_s	20/28	7/8	99.8	99.3	37.5	-0.7

Table 1: Selected structural budgets. CC and ρ are percentages; $\Delta\text{Inv-F1}$ is measured relative to Ω_{full} after full-data training.

Budget choice. We select the sparsest configuration that maintains performance comparable to Ω_{full} , obtaining $\Omega^* = \Omega_{20,3}$ for Q_l and $\Omega^* = \Omega_{20,7}$ for Q_s . Both configurations preserve nearly all cumulative layer contributions while assigning different head-group budgets according to model-specific contribution distributions⁵.

6 Results

All experiments in this section use Q_l trained on the full training set $\mathcal{D}$ and are evaluated on $\mathcal{T}_{\text{IND}}$ unless otherwise specified.

Cache Reuse Table 2 reports both the effectiveness and efficiency of resource-wise attention. We compare against Dense, and evaluate two incremental configurations: Block eliminates cross-resource attention while retaining the original positional indices, and Ω_{full} further reindexes positions to con-

⁵Appendix B presents detailed contribution weights and a preliminary budget analysis across model scales for identifying the optimal routing configuration Ω^* .

struct composition-independent resource blocks. Both configurations remain comparable to Dense, with all effectiveness metrics differing by at most 0.2%. Resource-wise attention also enables resource representations to be pre-encoded with a shared prefix and reused across requests, reducing online computation and allowing Ω_{full} to achieve a $3.655\times$ TTFT speedup over Dense. Consistent with findings from other tasks (Wang et al., 2021; Haviv et al., 2022; Kazemnejad et al., 2023), these results suggest that cross-resource interactions and global resource ordering provide limited benefits in this setting, while resource-local position re-indexing preserves prediction quality and enables efficient resource-level reuse.

Method	Inv-F1	ID-F1	Halluc.	TTFT (ms)
Dense	82.4	96.0	0.0	26.319
Block	82.2	95.8	0.1	—
Ω_{full}	82.3	96.0	0.2	7.200

Table 2: Resource-wise attention ablation.

Model Comparison Table 3 compares structural and semantic pruning strategies built on Ω_{full} on $\mathcal{T}_{\text{IND}}$, reporting both efficiency and invocation effectiveness. To assess robustness to distribution shift, Table 4 reports effectiveness on $\mathcal{T}_{\text{OOD}}$.

Under matched structural budgets, contribution-based selection consistently surpasses the corresponding baselines. At the 20-layer budget, $\Omega_{20,G}$ and $\Omega_{\text{full}} + \text{SPEED}$ (Oh et al., 2026) exhibit nearly identical efficiency, whereas $\Omega_{20,G}$ improves Inv-F1 by 3.2% on $\mathcal{T}_{\text{IND}}$ and 10.0% on $\mathcal{T}_{\text{OOD}}$. For joint layer-head-group pruning, $\text{SA}_{20,3}$ ranks routes by their attention mass over each resource and retains the top 20 layers and 3 KV head groups. Despite comparable efficiency, $\Omega_{20,3}$ improves Inv-

Method	Attn. $\times$	Mem. $\downarrow$	Inv-F1	ID-F1	Halluc.
Dense	—	—	82.4	96.0	0.0
Ω_{full}	$1.001\times$	0.47%	82.3	96.0	0.2
<i>Structural pruning</i>					
$\Omega_{20,G}$	$1.016\times$	44.71%	82.5	95.7	0.3
$\Omega_{\text{full}} + \text{SPEED}$	$1.013\times$	44.71%	79.3	92.9	2.8
$\Omega_{20,3}$	$1.314\times$	79.27%	82.1	95.6	0.2
$\Omega_{\text{full}} + \text{SA}_{20,3}$	$1.302\times$	79.27%	79.1	93.7	1.0
<i>Semantic pruning</i>					
$\Omega_{\text{full}} + \text{Gist}$	$1.029\times$	99.22%	39.2	46.9	51.4
$\Omega_{\text{full}} + \text{Beacon}$	$1.020\times$	75.42%	78.6	92.5	1.4
$\Omega_{\text{full}} + \text{SMP}$	$1.021\times$	63.57%	81.6	95.2	0.2
ReCache	$1.423\times$	92.43%	80.3	94.9	0.2

Table 3: Results on $\mathcal{T}_{\text{IND}}$. Efficiency is reported relative to Dense. SMP denotes Semantic Pruning.

Method	Inv-F1	ID-F1	Halluc.
Dense	66.3	95.6	0.0
Ω_{full}	64.7	94.4	0.4
<i>Structural pruning</i>			
$\Omega_{20,G}$	64.3	94.3	0.5
$\Omega_{\text{full}} + \text{SPEED}$	54.3	81.6	13.4
$\Omega_{20,3}$	63.2	93.3	0.5
$\Omega_{\text{full}} + \text{SA}_{20,3}$	58.2	88.4	5.1
<i>Semantic pruning</i>			
$\Omega_{\text{full}} + \text{Gist}$	9.7	17.8	79.9
$\Omega_{\text{full}} + \text{Beacon}$	58.5	89.7	4.2
$\Omega_{\text{full}} + \text{SMP}$	62.8	94.1	0.3
ReCache	60.8	92.8	0.6

Table 4: $\mathcal{T}_{\text{OOD}}$ Results. SMP denotes Semantic Pruning.

F1 over $\text{SA}_{20,3}$ by 3.0% and 5.0% points on the two test sets, respectively. The OOD hallucination rates further decrease from 13.4% to 0.5% relative to SPEED and from 5.1% to 0.5% relative to $\text{SA}_{20,3}$, demonstrating the stronger generalization of contribution-based routing.

For semantic pruning⁶, Gist (Mu et al., 2023) attains the largest standalone KV reduction but severely degrades invocation effectiveness, indicating that a single summary vector cannot preserve the fine-grained information required for resource invocation. Beacon (Zhang et al., 2024) retains more information through chunk-level summaries, yet remains less effective than field-aware Semantic Pruning and exhibits higher hallucination, particularly on $\mathcal{T}_{\text{OOD}}$. This gap indicates that partial coverage of critical fields is insufficient for robust generalization to unseen resources. Detailed field ablations are provided in Appendix C.

Combining all strategies, **ReCache** achieves the highest attention speedup ($1.423\times$) and a comparable KV reduction (92.43%) among configurations that retain competitive effectiveness. It retains Inv-F1 scores of 80.3% on $\mathcal{T}_{\text{IND}}$ and 60.8% on $\mathcal{T}_{\text{OOD}}$, corresponding to 97.5% and 91.8% of performance of Dense, respectively.

Efficiency Scaling We further evaluate efficiency across four ranges of total resource length $D_{\mathcal{R}}$. As shown in Figure 4, Dense TTFT increases from tens of milliseconds to over 5,000 ms as resource contexts expand, whereas Ω_{full} maintains TTFT below 50 ms through resource-level cache reuse. ReCache further reduces this prefill overhead, exhibiting an almost flat profile slightly above 5 ms across all ranges.

⁶For fair comparison, both baselines are adapted to the resource setting by assigning summary tokens to each resource, following their original token-level compression mechanism.

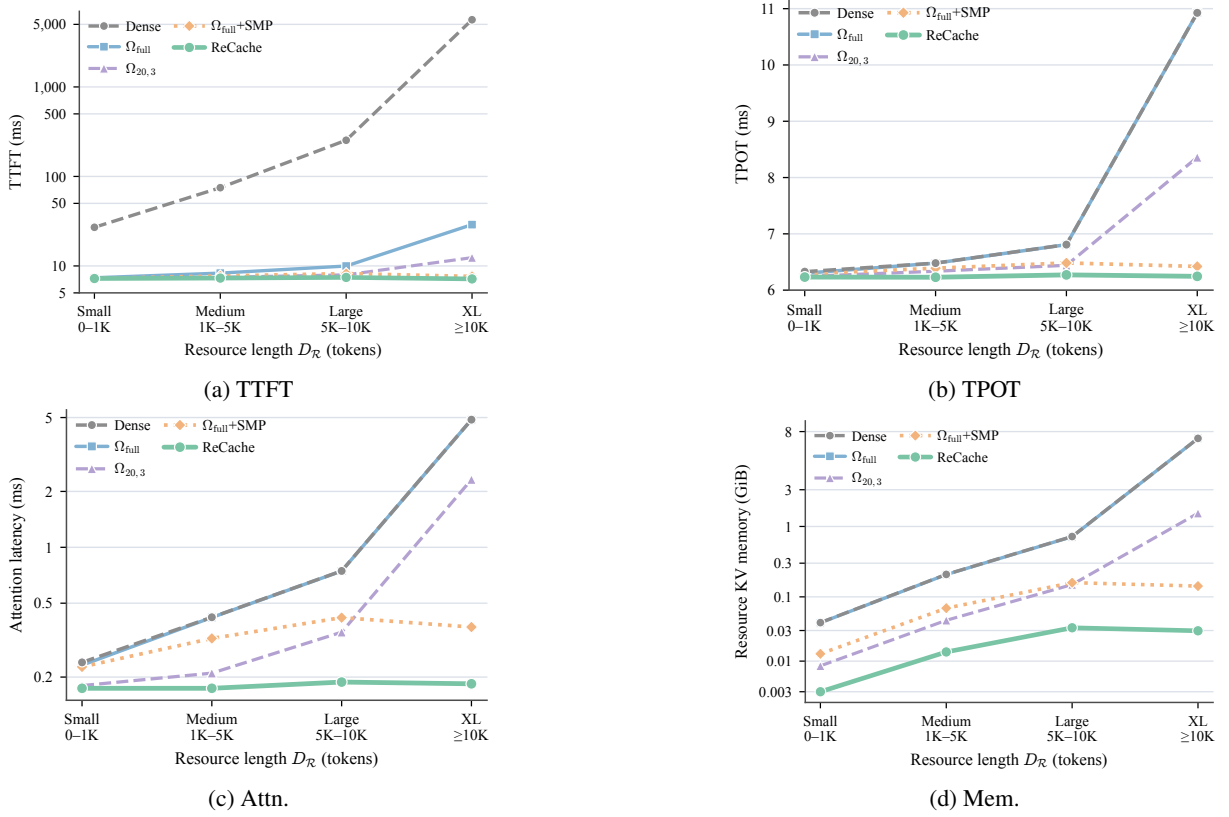


Figure 4: Efficiency on varying resource lengths D_R . Requests are grouped by average resource length into Small ($0 \leq D_R < 1K$), Medium ($1K \leq D_R < 5K$), Large ($5K \leq D_R < 10K$), and XL ($D_R \geq 10K$). SMP denotes semantic pruning.

Structural and semantic pruning further improve decoding efficiency. Dense and Ω_{full} reach approximately 11 ms TPOT and 5 ms attention latency, whereas ReCache maintains TPOT slightly above 6 ms and attention latency below 0.2 ms with consistently stable decoding latency across resource scales. At XL, semantic pruning ($\Omega_{full}+SMP$) reduces attention latency below 0.5 ms compared with over 2 ms under structural pruning ($\Omega_{20,3}$) alone, as longer resource contexts amplify the benefit of reducing tokens from semantic fields. Combining both pruning strategies therefore achieves the lowest and most stable latency.

ReCache also caps allocated Mem. at 0.03 GiB, whereas Dense and Ω_{full} approach 8 GiB as resource contexts grow. These bounded prefill, decoding, and memory costs demonstrate the suitability of ReCache for agentic workloads with increasingly large resource contexts.

7 Conclusion

We presented ReCache, a framework for reusing and compressing tool and skill schemas in agentic language models. Resource-wise attention encodes each complete schema as a resource-composition-

invariant KV block, eliminating repeated encoding across retrieval combinations. After construction, field-aware semantic pruning retains invocation-critical states, while contribution-guided structural pruning exposes them only to influential layer-KV-head-group routes. This separation preserves information during encoding while reducing the states stored and processed during inference.

Across seven tool- and skill-use datasets, independently reusable blocks match dense invocation quality and substantially reduced prefill latency. Combining structural and semantic pruning reduces allocated KV-tensor memory by 92.43% and accelerates attention by $1.423\times$, while retaining 97.5% of dense Inv-F1 in-distribution and 91.8% on unseen resources. The structural analysis further showed that effective routing budgets depend on model-scale redundancy, but can be selected with the same contribution-based criterion. ReCache therefore offers a practical way to scale dynamic resource contexts without repeatedly encoding or densely exposing every schema.

Limitations

Our evaluation focuses on two Qwen3 backbones, with end-to-end experiments primarily conducted on Qwen3-4B, as full-parameter fine-tuning of larger models was beyond the computational budget of our current infrastructure. While the optimal structural budget varies across model scales, extending the analysis to larger models and other architectures remains an important direction. Since ReCache modifies attention masks and positional layouts, the current study focuses on trainable settings, while adapting it to frozen models is left for future work. In addition, ReCache is designed for environments with relatively stable system instructions and resource schemas, where resource representations can be reused across different user queries and retrieval combinations. Extending reuse to highly dynamic resource environments, where retrieval order or cross-resource dependencies carry essential information, provides another avenue for future exploration.

References

- Xiaofan Bai, Hongqiang Lin, Chao Liu, Yantao Zhang, Xuan Jin, Xipeng Cao, and Yuhong Li. 2026. Skillzip: Evaluation-free skill compression for self-evolving agents by discovering reusable structure. *arXiv preprint arXiv:2608.11079*.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846.
- Zahra Dehghanighobadi and Asja Fischer. 2026. DepthKV: Layer-dependent kv cache pruning for long-context llm inference. *arXiv preprint arXiv:2604.24647*.
- Yiran Ding, Li Lina Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. LongRoPE: Extending LLM context window beyond 2 million tokens. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11091–11104. PMLR.
- Angela Fan, Edouard Grave, and Armand Joulin. 2020. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations*.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338.
- Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in neural information processing systems*, 36:45870–45894.
- Adi Haviv, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy. 2022. Transformer language models without positional encodings still learn positional information. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 1382–1390.
- Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. 2024. Epic: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*.
- Shouzheng Huang, Meishan Zhang, Baotian Hu, and Min Zhang. 2026. Toolomni: Enabling open-world tool use via agentic learning with proactive retrieval and grounded execution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37421–37439.
- Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*, pages 874–880.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmllingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 13358–13376.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. Longllmllingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677.
- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. 2026. SoK: Agentic skills – beyond tool use in LLM agents. *arXiv preprint arXiv:2602.20867*.
- Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. The impact of positional encoding on length generalization in transformers. *arXiv preprint arXiv:2305.19466*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

- Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-bank: A comprehensive benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianfu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-Chung Lee. 2026. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *CoRR*, abs/2602.12670.
- Xiaoqiang Lin, Aritra Ghosh, Bryan Kian Hsiang Low, Anshumali Shrivastava, and Vijai Mohan. 2025. Re-frag: Rethinking rag based decoding. *arXiv preprint arXiv:2509.01092*.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Ning Wu, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. 2024a. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*.
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Bo Li, Xuming Hu, and Xiaowen Chu. 2026. Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference. *Advances in Neural Information Processing Systems*, 38:28728–28778.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh Murthy, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan Wang, Shelby Heinecke, and Caiming Xiong. 2024b. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482.
- Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.
- Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, volume 32.
- Jesse Mu, Xiang Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352.
- Niklas Muennighoff. 2022. Sgpt: Gpt sentence embeddings for semantic search. *arXiv preprint arXiv:2202.08904*.
- Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow prefill, deep decoding: Efficient long-context inference via layer-asymmetric kv visibility. *arXiv preprint arXiv:2605.06105*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551.
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468.
- Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. 2025. Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models. *arXiv preprint arXiv:2503.01763*.
- Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, Ye Tian, and Sujian Li. 2023. RestGPT: Connecting large language models with real-world RESTful APIs. *arXiv preprint arXiv:2306.06624*.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Elena Voita, David Talbot, Fedor Moiseev, Rico Senrich, and Ivan Titov. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808.

- Benyou Wang, Lifeng Shang, Christina Lioma, Xin Jiang, Hao Yang, Qun Liu, and Jakob Grue Simonsen. 2021. [On position embeddings in {bert}](#). In *International Conference on Learning Representations*.
- Jun Wang, Jiamu Zhou, Muning Wen, Xiaoyun Mo, Haoyu Zhang, Qiqiang Lin, Cheng Jin, Xihuai Wang, Weinan Zhang, Qiying Peng, and Jun Wang. 2024a. HammerBench: Fine-grained function-calling evaluation in real mobile device scenarios. *arXiv preprint arXiv:2412.16516*.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024b. Improving text embeddings with large language models. *arXiv preprint arXiv:2401.00368*.
- Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2025. Toolgen: Unified tool retrieval and calling via generation. In *International Conference on Learning Representations*, volume 2025, pages 73473–73498.
- Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. 2024. [Retrieval head mechanistically explains long-context factuality](#). *arXiv preprint arXiv:2404.15574*.
- Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2025. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. In *International Conference on Learning Representations*, volume 2025, pages 37228–37253.
- Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*.
- Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Pooven-dran, and Rameswar Panda. 2025. TOU-CAN: Synthesizing 1.5m tool-agentic data from real-world MCP environments. *arXiv preprint arXiv:2510.01179*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025a. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Chen Yang, Ran Le, Yun Xing, Zhenwei An, Zongchao Chen, Wayne Xin Zhao, Yang Song, and Tao Zhang. 2025b. ToolMind technical report: A large-scale, reasoning-enhanced tool-use dataset. *arXiv preprint arXiv:2511.15718*.
- Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025c. Kvlink: Accelerating large language models via efficient kv cache reuse. *arXiv preprint arXiv:2502.16002*.
- Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*, pages 94–109.
- Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. 2025. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *arXiv preprint arXiv:2510.12872*.
- Lingfan Yu, Jinkun Lin, and Jinyang Li. 2025. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 144–158.
- Peijie Yu, Wei Liu, Yifan Yang, Jinjian Li, Zelong Zhang, Xiao Feng, and Feng Zhang. 2026. Benchmarking llm tool-use in the wild. *arXiv preprint arXiv:2604.06185*.
- Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. 2024. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*.
- Quqing Zhang, Kai Chen, Ning Liao, Zehao Lin, Bo Tang, Feiyu Xiong, Zhiyu Li, and Xiaoxing Wang. 2026. Sparsex: Efficient segment-level kv cache sharing for interleaved llm serving. *arXiv preprint arXiv:2606.01751*.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. [H2O: heavy-hitter oracle for efficient generative inference of large language models](#). In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583.

Yanzhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. [Skillrouter: Skill routing for LLM agents at scale](#). *CoRR*, abs/2603.22455.

Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2024. Promptreps: Prompting large language models to generate dense and sparse representations for zero-shot document retrieval. *arXiv preprint arXiv:2404.18424*.

A Dataset Construction Details

This appendix describes how the heterogeneous source pools are converted into the training and evaluation data used by ReCache. The objective is to obtain broad resource-use coverage without allowing invalid traces or repeated resource configurations to confound the evaluation of resource compression. Table 5 reports the source-level statistics before and after construction.

Problems in direct aggregation. We combine ToolACE (Liu et al., 2024a), APIGEN (Liu et al., 2024b), ToolMind (Yang et al., 2025b), Toucan (Xu et al., 2025), ToolRet (Shi et al., 2025), WildToolBench (Yu et al., 2026), and SkillRouter (Zheng et al., 2026) because they cover complementary schema conventions, candidate-set sizes, interaction lengths, and resource types. Directly concatenating them, however, exposes two systematic problems. First, their declarations and invocations use heterogeneous formats, and generated trajectories may contain unparsable arguments, no actual invocation, or hallucinated invocations. Second, their resource distributions are highly repetitive. Across the five sources with frequency statistics, 29,232 of 37,824 source-level distinct resource names recur in multiple records (77.3%), while 9,901 of 52,621 candidate records repeat an exact previously observed candidate subset (18.8%). The imbalance is concentrated rather than uniform: recurring-name rates reach 99.96% in APIGEN, 100% in Toucan, and 99.95% in ToolMind, while 52.8% of Toucan records duplicate an existing candidate subset. Consequently, random downsampling would preserve shortcuts based on familiar resource identities and candidate configurations instead of encouraging resource-conditioned invocation.

Two-stage diversity-first pipeline. Before sampling, we normalize resource names by trimming whitespace and case, repair lightweight JSON-format errors, parse every declared resource and assistant invocation, and discard traces with no usable invocations, unresolved invocations, undeclared resource names, or sequences beyond the context budget. Repeated invocations to the same resource inside one trajectory are counted once, preventing retries from inflating frequency estimates; when source-provided quality labels are available, they are applied before sampling, including the realism, verifiability, stability, completeness, and conciseness filters for Toucan. The diversity sampler then

proceeds in two stages. For each valid example x , we define the candidate-configuration signature

$$\sigma(x) = \text{sort}\{n(R) : R \in \mathcal{R}(x)\},$$

where $\mathcal{R}(x)$ is its declared resource set and $n(R)$ is the normalized resource name.

In Stage I, an example is accepted only if its signature has not appeared and none of its called resources has been used by a previously selected example, jointly prioritizing candidate-subset and invocation-level diversity. Since strict uniqueness cannot fill every source quota, Stage II admits additional valid examples in the same quality-ranked order until each predefined quota is reached. This second stage preserves source and scenario coverage while retaining the diversity gains established by the first pass. A final global validity pass reduces the 52,964 sampled candidates to 49,424 training examples, removing 3,540 records (6.7%).

Evaluation split construction. After the two-stage selection, we construct in-distribution (IND) and out-of-distribution (OOD) candidate pools at the resource level. For each resource R_i , $\text{frq}(R_i)$ is the number of valid examples in which it is declared; low-frequency resources are reserved for OOD evaluation and removed from training. An example is eligible for the OOD pool only when every declared resource belongs to this reserved set, which prevents partial overlap with training resources. IND candidates use resources observed during training but contain different examples, queries, and invocation patterns. Exact configuration signatures are deduplicated independently within both pools before sampling, preventing repeated candidate subsets from making either test condition artificially easy. We finally sample 1,000 IND and 1,000 OOD examples, yielding matched evaluation sizes while isolating generalization to unseen resources.

B Structural Capacity Analysis

B.1 Preliminary Budget Investigation

We investigate how computation is allocated across transformer layers and KV head groups under different model scales and training regimes when identifying the optimal structural budget Ω^* . The analysis focuses on two factors that influence the achievable sparsity level: the amount of optimization supervision and the representational capacity of the backbone model.

Group	Source	Train pool	Distinct names	Subset dup.	Recurring names	IND cand.	OOD cand.
Tool	ToolACE	8,400	14,566	178	7,144	200	195
	APIGEN	10,000	2,806	734	2,805	500	146
	Toucan (all generators)	13,071	4,355	6,901	4,355	600	493
	- Kimi-K2	3,333	1,103	1,772	1,103	200	145
	- GPT-OSS-120B	4,184	1,412	2,162	1,412	200	130
	- Qwen3-32B	5,554	1,840	2,967	1,840	200	218
	ToolRet	1,150	1,593	235	431	100	150
	ToolMind	20,000	14,504	1,853	14,497	500	190
	WildToolBench	256	—	—	—	—	—
Skill	SkillRouter	87	—	—	—	—	—
Sources with frequency statistics		52,621	37,824	9,901	29,232	1,900	1,174
Candidate pool		52,964	—	—	—	1,900	1,174
Final dataset		49,424	—	—	—	1,000	1,000

Table 5: Dataset composition and split statistics. Source rows report eligible examples before final selection; the Toucan aggregate is not double-counted in the summary row. **Distinct names** counts source-level unique declared tools or skills, **Subset dup.** counts records beyond the first occurrence of an identical candidate configuration, and **Recurring names** counts resources appearing in multiple records. The summary row corresponds to 18.8% duplicated candidate records and 77.3% recurring resource names. IND cand. and OOD cand. denote eligible in-distribution and out-of-distribution candidates, and dashes mark unavailable statistics.

Observation 1: Head-group pruning is more sensitive to limited supervision. Under $\mathcal{D}_{\text{subset}}$, Q_s is substantially more sensitive to head-group sparsification than to layer sparsification. Retaining six of eight KV head groups across all layers, denoted by $\Omega_{L,6}$, reduces Inv-F1 by 18.7% compared to Ω_{full} (Table 6). In contrast, retaining 20 of 28 layers across all head groups, denoted by $\Omega_{20,G}$, changes Inv-F1 by less than 0.2%. This asymmetry indicates that head-group sparsification requires stronger supervision to identify effective resource-conditioned computation patterns.

The auxiliary-loss experiment provides further evidence. Incorporating an invocation-oriented objective for Q_s with $\Omega_{L,6}$ under $\mathcal{D}_{\text{subset}}$ increases Inv-F1 to 69.9%, indicating that direct invocation supervision partially mitigates the degradation caused by head-group sparsification. Similarly, training on the full dataset $\mathcal{D}$ narrows the performance gap between sparse configurations and Ω_{full} . Together, these results show that the viable structural budget depends jointly on retained route capacity and the available optimization signal.

Observation 2: Model scale affects the required structural budget. Despite full-data training, models with different capacities retain distinct structural requirements (Table 7). Q_l maintains comparable invocation performance with fewer active head groups, while Q_s requires broader head-group participation. This implies that larger models can sustain resource-conditioned computation with

more aggressive structural sparsification.

Overall, these observations suggest that the attainable structural sparsity is determined by the effective capacity of retained resource-visible routes, which depends on both optimization supervision and model representation capability. The larger Qwen3-4B provides greater capacity within each retained route, attributable to its larger hidden and FFN dimensions and its four query heads per KV group, compared with two in Qwen3-1.7B.

B.2 Contribution Distribution

Table 9 reports detailed weights of individual units, the cumulative contribution coverage of layers, and head groups, respectively.

C Semantic Pruning Ablation

Table 8 presents the semantic pruning ablation on $\mathcal{D}_{\text{subset}}$, where all experiments are conducted under $\Omega_{20,3}$. The objective is to identify the semantic fields that should be retained before scaling the selected configuration to full-data training.

The results reveal a clear hierarchy among the evaluated semantic fields. Adding only the resource name already yields substantial improvements, reducing the hallucination rate by 97.63% relative to the suffix-only setting while increasing all resource-ID metrics above 88%. Incorporating argument names further improves Inv-F1 by 22.5%, demonstrating that parameter identifiers provide essential grounding for argument generation.

$\mathcal{D}_{\text{subset}}$						$\mathcal{D}$					
Config	Inv-F1	Resource ID			Halluc.	Config	Inv-F1	Resource ID			Halluc.
		ID-P	ID-R	ID-F1				ID-P	ID-R	ID-F1	
Ω_{full}	72.1	92.0	90.8	90.9	0.4	Ω_{full}	80.7	94.9	94.3	94.4	0.1
$\Omega_{L,5}$	49.6	74.5	73.5	73.4	4.8	$\Omega_{L,5}$	79.5	94.4	93.6	93.8	0.6
$\Omega_{L,6}$	53.4	79.6	77.9	78.3	3.2	$\Omega_{L,6}$	79.8	94.4	93.9	93.9	0.3
$\Omega_{L,7}$	57.3	83.4	81.8	82.1	2.9	$\Omega_{L,7}$	80.6	94.8	94.2	94.3	0.2
$\Omega_{15,G}$	69.3	89.2	88.3	88.3	1.1	$\Omega_{15,G}$	80.0	94.2	93.7	93.8	0.3
$\Omega_{20,G}$	70.5	90.6	89.5	89.6	1.1	$\Omega_{20,G}$	80.6	94.7	94.1	94.2	0.4

Table 6: Structural budget ablation for Q_s across training data regimes with comparable budgets.

Config	Inv-F1	Resource ID			Halluc.
		ID-P	ID-R	ID-F1	
Q_s					
Ω_{full}	80.7	94.9	94.3	94.4	0.1
$\Omega_{20,5}$	78.8	94.1	93.5	93.6	0.5
$\Omega_{20,7}$	80.0	94.7	94.0	94.1	0.3
Q_t					
Ω_{full}	82.3	96.4	95.8	96.0	0.2
$\Omega_{20,2}$	81.9	95.3	94.7	94.7	0.3
$\Omega_{20,3}$	82.1	96.1	95.5	95.6	0.2

Table 7: Structural budget ablation trained on $\mathcal{D}$.

Config	Inv-F1	Resource ID			Halluc.
		ID-P	ID-R	ID-F1	
$\Omega_{20,3}$	76.1	93.1	92.8	92.5	0.9
<i>Schema representation ablation</i>					
Suffix-only Schema	14.8	22.8	23.0	22.9	76.0
+ Resource Name	45.2	89.9	88.8	88.9	1.8
+ Arg Name	67.7	92.1	91.1	91.2	1.3
+ Resource Desc.	67.9	92.2	91.4	91.4	1.1
+ Arg Desc.	72.8	92.8	91.9	92.0	1.0
+ Both Desc.	73.4	93.0	92.2	92.2	1.2

Table 8: Semantic pruning ablation trained on $\mathcal{D}_{\text{subset}}$. Arg stands for argument, Desc. stands for description. $\Omega_{20,3}$ denotes the routing configuration with $K_L = 20, K_G = 3$.

Within the considered schema components, argument descriptions provide additional benefits by supplying semantic constraints for parameter prediction. In contrast, resource descriptions offer limited contribution. Adding only resource descriptions on top of resource and argument names yields negligible improvement, while additionally including resource descriptions alongside argument descriptions changes all evaluation metrics by less than 0.7% compared with preserving argument descriptions alone. These results suggest that resource names, argument names, and argument descriptions constitute the core semantic components for resource invocation, while resource descriptions provide limited additional information once the executable interface has been preserved.

Layers							
$Q_l(\text{Qwen3-4B})$				$Q_s(\text{Qwen3-1.7B})$			
rank	unit	w_i (%)	CC (%)	rank	unit	w_i (%)	CC (%)
1	L_{34}	15.56	15.6	1	L_{22}	20.09	20.1
2	L_{35}	10.53	26.1	2	L_{25}	12.90	33.0
3	L_{23}	9.37	35.5	3	L_{20}	12.14	45.1
4	L_{25}	8.31	43.8	4	L_{26}	10.90	56.0
5	L_{32}	8.01	51.8	5	L_{27}	10.39	66.4
6	L_{30}	6.95	58.7	6	L_{17}	6.82	73.2
7	L_{33}	6.89	65.6	7	L_{21}	6.70	79.9
8	L_{27}	5.17	70.8	8	L_{19}	4.65	84.6
9	L_{21}	4.87	75.7	9	L_{24}	4.00	88.6
10	L_{28}	4.64	80.3	10	L_{23}	2.13	90.7
11	L_{24}	4.02	84.3	11	L_{18}	1.89	92.6
12	L_{22}	3.74	88.1	12	L_{15}	1.73	94.3
13	L_{17}	2.30	90.4	13	L_{13}	1.39	95.7
14	L_{31}	1.96	92.3	14	L_{10}	1.13	96.9
15	L_{18}	1.29	93.6	15	L_9	1.03	97.9
16	L_{14}	1.26	94.9	16	L_4	0.88	98.8
17	L_{20}	0.78	95.7	17	L_{16}	0.56	99.3
18	L_{36}	0.69	96.3	18	L_{12}	0.18	99.5
19	L_{19}	0.68	97.0	19	L_{11}	0.15	99.7
20	L_{16}	0.68	97.7	20	L_8	0.14	99.8
21	L_{10}	0.60	98.3	21	L_{28}	0.11	99.9
22	L_5	0.33	98.6	22	L_5	0.06	100.0
23	L_{11}	0.29	98.9	23	L_2	0.02	100.0
24	L_9	0.27	99.2	24	L_1	0.00	100.0
25	L_{12}	0.26	99.5	25	L_3	0.00	100.0
26	L_{15}	0.26	99.7	26	L_6	0.00	100.0
27	L_{13}	0.13	99.8	27	L_7	0.00	100.0
28	L_8	0.07	99.9	28	L_{14}	0.00	100.0
29	L_3	0.07	100.0	—	—	—	—
30	L_1	0.00	100.0	—	—	—	—
31	L_2	0.00	100.0	—	—	—	—
32	L_4	0.00	100.0	—	—	—	—
33	L_6	0.00	100.0	—	—	—	—
34	L_7	0.00	100.0	—	—	—	—
35	L_{26}	0.00	100.0	—	—	—	—
36	L_{29}	0.00	100.0	—	—	—	—

KV head groups							
$Q_l(\text{Qwen3-4B})$				$Q_s(\text{Qwen3-1.7B})$			
rank	unit	w_i (%)	CC (%)	rank	unit	w_i (%)	CC (%)
1	G_4	16.54	16.5	1	G_5	22.84	22.8
2	G_6	16.17	32.7	2	G_3	19.01	41.9
3	G_3	14.64	47.3	3	G_8	17.27	59.1
4	G_2	12.37	59.7	4	G_6	16.77	75.9
5	G_8	11.62	71.3	5	G_1	10.91	86.8
6	G_1	9.68	81.0	6	G_7	8.83	95.6
7	G_5	9.61	90.6	7	G_4	3.64	99.3
8	G_7	9.37	100.0	8	G_2	0.74	100.0

Table 9: Individual contribution weights and cumulative contribution coverage of structural units. Within each panel, units are ranked independently by decreasing w_i , and CC at rank k is the cumulative positive contribution weight through rank k . Bold entries fall within the empirical budgets $K_L=20$, $K_G=3$ for Q_l and $K_L=20$, $K_G=7$ for Q_s .